

实验报告

系统开发工具基础 第 4 周

姓 名： 王本熙

学 号： 25020007117

日 期： 2026 年 9 月 14 日

平 台： Windows + WSL (Ubuntu)

目录

1 实验目的	2
2 实验环境	2
3 q13: 建立可执行的本地质量门禁	2
3.1 题目要求	2
3.2 实验步骤	3
3.3 运行结果与截图	4
3.4 结果分析	4
4 q14: 让 Make 只重建真正受影响的产物	6
4.1 题目要求	6
4.2 实验步骤	6
4.3 运行结果与截图	7
4.4 结果分析	7
5 q15: 把本地 API 数据转换为可读报告	7
5.1 题目要求	7
5.2 实验步骤	8
5.3 运行结果与截图	9
5.4 结果分析	9
6 q16: 修复并交付一个陌生的小型工具仓库	9
6.1 题目要求	9
6.2 实验步骤	9
6.3 运行结果与截图	11
6.4 结果分析	11
7 第 4 周课后练习	13
7.1 Makefile 的 clean 目标	13
7.2 Rust 依赖版本语法的几种场景	13
7.3 Git pre-commit 钩子: 提交前自动构建	13
7.4 GitHub Pages、Actions 与自定义 Action	14
7.5 格式化器、linter 与 pre-commit 钩子的配合	14
7.6 测试覆盖率与 AI agent	15
7.7 持续集成: GitHub Actions	15
7.8 grep、正则与 semgrep	16
7.9 IDE 中的正则查找替换	16
7.10 捕获 JSON 中 name 的正则	16

7.11 更推荐的做法：直接用 JSON 解析器	17
8 代码与仓库	17
8.1 Git 提交记录	17
9 结论	18

第 1 章 实验目的

本实验旨在掌握以下内容：

- 建立可执行的本地质量门禁：`ruff format --check`、`ruff check`、`pytest` 串联为 `check.sh`，验证返回 0（q13）；
- 编写 Makefile 实现增量构建：准确声明数据文件、脚本和中间产物的依赖关系，`all` 与 `clean` 声明为 `.PHONY`（q14）；
- 使用本地 HTTP 服务 + `curl` + `jq` 筛选 JSON 数据并生成 Markdown 报告（q15）；
- 综合测试：临时改坏实现确认测试失败，再定位修复，编写含 `check`、`build`、`clean` 的 Makefile，计算 wheel 的 SHA-256 并提交为单一 Git 提交（q16）。

第 2 章 实验环境

表 1: 实验环境

项目	配置
操作系统	Windows + WSL (Ubuntu 24.04)
Shell	bash
Python	3.12.3
编辑器	VS Code / Trae IDE / micro
版本管理	Git
远程仓库	GitHub
构建工具	Make / python -m build
静态检查	ruff (format + check)
测试	pytest
JSON 处理	jq 1.7.1

第 3 章 q13: 建立可执行的本地质量门禁

3.1 题目要求

主题：代码质量。复用第 10 题完成的 `greetlab` 包，在 q13 中建立一个轻量本地质量检查：

- 复制 q10 为 q13；在 `pyproject.toml` 中加入 `ruff` 和 `pytest` 的最小配置；
- 补充一个正常姓名测试和一个空白姓名测试，每个测试包含有效断言；
- 运行 `ruff format`、`ruff check` 和 `pytest`，修复全部问题，不得全局忽略规则；

- 编写 `check.sh`, 依次执行 `ruff format --check`、`ruff check` 和 `pytest`; 运行并保证返回 0。

3.2 实验步骤

复制 q10 为 q13, 更新 `pyproject.toml` 加入 `ruff` 与 `pytest` 配置:

```
1 [tool.ruff]
2 line-length = 88
3 target-version = "py312"
4
5 [tool.ruff.lint]
6 select = ["E", "F", "I", "W"]
7
8 [tool.ruff.format]
9 quote-style = "double"
10
11 [tool.pytest.ini_options]
12 testpaths = ["tests"]
```

两个测试:

```
1 # tests/test_cli.py
2 import sys
3 import pytest
4 from greetlab.cli import main
5
6 def test_normal_name(monkeypatch, capsys):
7     monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "World"])
8     main()
9     captured = capsys.readouterr()
10    assert captured.out.strip() == "Hello, World!"
11
12 def test_blank_name_raises_system_exit(monkeypatch, capsys):
13     monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", ""])
14     with pytest.raises(SystemExit) as excinfo:
15         main()
16     assert excinfo.value.code == 2
17     captured = capsys.readouterr()
18     assert captured.out == ""
```

`check.sh`:

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 echo "=== ruff format check ==="
5 ruff format --check .
```

```
6
7 echo "=== ruff check ==="
8 ruff check .
9
10 echo "=== pytest ==="
11 pytest
12
13 echo "=== all checks passed ==="
```

运行验证：

```
1 cd q13
2 python3 -m venv venv && source venv/bin/activate
3 pip install -e . ruff pytest
4 bash check.sh
5 # === ruff format check ===
6 # 3 files already formatted
7 # === ruff check ===
8 # All checks passed!
9 # === pytest ===
10 # collected 2 items
11 # tests/test_cli.py .. [100%]
12 # 2 passed in 0.01s
13 # === all checks passed ===
```

3.3 运行结果与截图

图 1 展示了 pytest 测试通过、环境配置与 check.sh 完整运行。

3.4 结果分析

- **pyproject.toml 配置:** [tool.ruff] 指定 line-length = 88 (PEP 8 默认) 和 target-version = "py312"; [tool.ruff.lint] select = ["E", "F", "I", "W"] 启用 pycodestyle 错误、pyflakes、isort、pycodestyle 警告; [tool.pytest.ini_options] testpaths = ["tests"] 让 pytest 自动发现测试;
- **两个测试:** test_normal_name 验证正常输入输出 Hello, World!; test_blank_name 验证空白姓名抛 SystemExit(2)。两者都使用 monkeypatch 注入 sys.argv、capsys 捕获 stdout, 隔离且无需真实命令行;
- **check.sh 串联:** set -euo pipefail 保证任何环节失败即退出, 不会出现“ruff check 失败但 pytest 仍跑”的情况; ruff format --check (仅检查不改写)、ruff check (静态分析)、pytest (行为验证) 三层防护, 返回值 0 即全部通过;
- **src-layout 陷阱:** 运行 pytest 前必须 pip install -e . 以可编辑模式安装包, 否则会报 ModuleNotFoundError: No module named 'greetlab'。

```

==== 1 failed, 1 passed in 0.01s ====
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q13$ micro tests/test_cli.py
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q13$ python -m pytest
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/wangbenxi/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py ..                                     [100%]

===== 2 passed in 0.01s =====
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q13$

```

(a) pytest: collected 2 items, 2 passed in 0.01s

```

(venv) wangbenxi@WIN-56NAAMF7VQA:~/q13$ deactivate
cd /home/wangbenxi/q13
rm -rf venv
wangbenxi@WIN-56NAAMF7VQA:~/q13$ python3 -m venv venv
source venv/bin/activate
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q13$ head -5 src/greetlab/cli.py
import argparse
import sys

def main():
    p = argparse.ArgumentParser()
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q13$ micro src/greetlab/cli.py
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q13$ micro pyproject.toml
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q13$ pip config set global.index-url https://
ypi.tuna.tsinghua.edu.cn/simple

```

(b) 创建 venv、编辑 cli.py 与 pyproject.toml、pip 配置清华源

```

Round 1 error (1 fixed, 0 remaining):
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q13$ ./check.sh
=== ruff format check ===
3 files already formatted
=== ruff check ===
All checks passed!
=== pytest ===
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/wangbenxi/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py ..                                     [100%]

===== 2 passed in 0.01s =====
=== all checks passed ===

```

(c) ./check.sh: ruff format check → ruff check → pytest → all checks passed

图 1: q13: 可执行的本地质量门禁

第 4 章 q14: 让 Make 只重建真正受影响的产物

4.1 题目要求

主题：元编程——构建系统。在 q14 中按给定内容创建三个源文件，只需完成 Makefile：

- 创建给定的 data.csv、stats.py、build_report.py 和 report.md；
- 编写 Makefile，包含 all、stats.txt、report.txt 和 clean；准确列出数据文件、脚本和中间产物依赖，并把 all、clean 声明为.PHONY；
- 验证首次 make 生成两个产物；第二次无改动时不执行配方；touch data.csv 后再次 make，确认 stats.txt 和 report.txt 重新生成；clean 只删除生成物。

4.2 实验步骤

按题目给定内容创建四个源文件：

```
1 # data.csv
2 name,value
3 alpha,2
4 beta,3
5 gamma,5
6 # stats.py / build_report.py / report.md 按题目给定
```

Makefile:

```
1 .PHONY: all clean
2
3 all: report.txt
4
5 stats.txt: data.csv stats.py
6     python3 stats.py
7
8 report.txt: stats.txt build_report.py report.md
9     python3 build_report.py
10
11 clean:
12     rm -f stats.txt report.txt
```

增量构建验证：

```
1 cd q14
2 make          # 首次: python3 stats.py → python3 build_report.py
3 make          # 二次: Nothing to be done for 'all'
4 touch data.csv
5 make          # 三次: data.csv 时间戳变了，两个产物都重建
6 make clean    # rm -f stats.txt report.txt
```

4.3 运行结果与截图

图 2 展示了增量构建的完整验证。

```
=== all checks passed ===
(wenv) wangbenxi@WIN-56NAAMF7VQA:~/q13$ cd /home/wangbenxi
mkdir q14 && cd q14
(wenv) wangbenxi@WIN-56NAAMF7VQA:~/q14$ micro data.csv
(wenv) wangbenxi@WIN-56NAAMF7VQA:~/q14$ micro stats.py
(wenv) wangbenxi@WIN-56NAAMF7VQA:~/q14$ micro build_report.py
(wenv) wangbenxi@WIN-56NAAMF7VQA:~/q14$ micro report.md
(wenv) wangbenxi@WIN-56NAAMF7VQA:~/q14$ micro Makefile
(wenv) wangbenxi@WIN-56NAAMF7VQA:~/q14$ make
```

(a) 创建 data.csv、stats.py、build_report.py、report.md、Makefile

```
python3 stats.py
python3 build_report.py
make: Nothing to be done for 'all'.
python3 stats.py
python3 build_report.py
rm -f stats.txt report.txt
```

(b) 首次 make 生成两产物 → 二次 make: Nothing to be done → touch data.csv 后重建 → make clean 删除生成物

图 2: q14: Makefile 增量构建验证

4.4 结果分析

- **依赖链正确:** all → report.txt → stats.txt → data.csv + stats.py, report.txt 额外依赖 build_report.py 和 report.md; Make 通过比较目标文件与依赖文件的修改时间戳决定是否重建;
- **.PHONY 声明:** .PHONY: all clean 告诉 Make all 和 clean 是命令名而非目标文件名, 即使当前目录下存在同名文件也会执行对应配方;
- **增量验证:** 首次 make 依次运行 python3 stats.py 和 python3 build_report.py; 二次 make 无任何文件改动, 输出 Nothing to be done for 'all'; touch data.csv 只更新 data.csv 的时间戳, Make 检测到 stats.txt 比依赖旧, 重建 stats.txt, 进而 report.txt 的依赖也变新, 联动重建;
- **中间产物清理:** clean: rm -f stats.txt report.txt 只删除目标文件, 保留源码, 符合 make clean 的通用约定。

第 5 章 q15: 把本地 API 数据转换为可读报告

5.1 题目要求

主题: 大杂烩——API、jq、CLI 约定与 Markdown。在 q15 中创建给定 JSON 和本地 HTTP 服务, 再用 curl 与 jq 生成简短 Markdown 报告:

- 把给定 JSON 保存为 `packages.json`，在 `q15` 目录运行 `python -m http.server 8000`;
- 使用 `curl -fsS` 获取 `http://127.0.0.1:8000/packages.json`;
- 使用 `jq` 筛选 `status` 为 `active` 且 `downloads` 不少于 100 的记录，并按 `downloads` 降序、`name` 升序排列;
- 编写 `api_report.sh`，把结果生成 `summary.md`，包含标题和 `name/version/downloads` 三列表格。

5.2 实验步骤

`packages.json` 按题目给定内容保存。HTTP 服务后台启动：

```
1 cd q15
2 python3 -m http.server 8000 &    # 后台启动 HTTP 服务
3 sudo apt install -y jq          # 安装 jq (WSL 默认未装)
```

`api_report.sh`:

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 curl -fsS http://127.0.0.1:8000/packages.json \
5 | jq -s '[] | select(.status == "active" and .downloads >= 100)] | sort_by(-.
   downloads, .name)' \
6 | jq -r '
7     "# Active Packages (downloads 100)",
8     "",
9     "| Name | Version | Downloads |",
10    "|-----|-----|-----|",
11    (.[] | "| \(.name) | \(.version) | \(.downloads) |")
12    ' > summary.md
13
14 cat summary.md
```

运行：

```
1 chmod +x api_report.sh
2 ./api_report.sh
3 # [14/Sep/2026 16:32:58] "GET /packages.json HTTP/1.1" 200 -
4 cat summary.md
5 # # Active Packages (downloads 100)
6 #
7 # | Name | Version | Downloads |
8 # |-----|-----|-----|
9 # | delta | 0.9.0 | 450 |
10 # | gamma | 1.5.1 | 450 |
11 # | alpha | 1.2.0 | 120 |
```

5.3 运行结果与截图

图 3 展示了 HTTP 服务启动、jq 安装与 `api_report.sh` 输出。

5.4 结果分析

- **curl -fsS**: `-f` 失败时静默退出码非零（不输出 HTML 错误页）；`-s` 静默模式不显示进度；`-S` 但仍在失败时输出错误信息；三者组合适合脚本中调用——成功无输出、失败有诊断；
- **jq 管道**: 第一个 jq 用 `-s (slurp)` 把数组包成单元素，再用 `select(.status == "active" and .downloads >= 100)` 筛选、`sort_by(-.downloads, .name)` 排序（负号 = 降序，字段名 = 升序）；第二个 jq 用 `-r (raw output)` 去掉 JSON 引号，用字符串模板直接拼 Markdown 表格；
- **筛选结果验证**: 5 个包中，`beta (inactive)`、`epsilon (downloads=80)` 被排除，剩下 `delta (450)`、`gamma (450)`、`alpha (120)`，`delta` 和 `gamma` 同 `downloads` 时按 `name` 升序（`delta < gamma`），正确；
- **jq 安装**: WSL Ubuntu 24.04 默认未装 jq，需 `sudo apt install -y jq`，注意 `-y` 跳过交互确认。

第 6 章 q16: 修复并交付一个陌生的小型工具仓库

6.1 题目要求

主题：课堂综合测试。复用第 13 题的项目，在 q16 中完成一次 15 分钟综合检查：

- 复制 q13 为 q16 并初始化/继续使用 Git；
- 把问候语实现临时改成始终返回字面量 `Hello, name!`，运行 `check.sh` 确认测试失败；随后定位并修复；
- 编写最小 Makefile，包含 `check`、`build` 和 `clean`；`check` 调用 `check.sh`，`build` 生成 `wheel`；
- 运行 `make check` 和 `make build`，计算 `wheel` 的 SHA-256，并把最终改动提交为一个内容聚焦的 Git 提交。

6.2 实验步骤

复制 q13 为 q16，初始化 Git：

```
1 cp -r q13 q16 && cd q16 && git init
```

临时改坏 `cli.py`：

```
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q15$ sudo apt install -y jq
[sudo] password for wangbenxi:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libjq1 libonig5
The following NEW packages will be installed:
  jq libjq1 libonig5
0 upgraded, 3 newly installed, 0 to remove and 0 not upgraded.
Need to get 380 kB of archives.
After this operation, 1127 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 libonig5 amd64 6.9.9-1build1 [172 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libjq1 amd64 1.7.1-3ubuntu0.24.04.2 [142 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 jq amd64 1.7.1-3ubuntu0.24.04.2 [65.7 kB]
Fetched 380 kB in 3s (133 kB/s)
Selecting previously unselected package libonig5:amd64.
(Reading database ... 56324 files and directories currently installed.)
Preparing to unpack .../libonig5_6.9.9-1build1_amd64.deb ...
Unpacking libonig5:amd64 (6.9.9-1build1) ...
Selecting previously unselected package libjq1:amd64.
Preparing to unpack .../libjq1_1.7.1-3ubuntu0.24.04.2_amd64.deb ...
Unpacking libjq1:amd64 (1.7.1-3ubuntu0.24.04.2) ...
Selecting previously unselected package jq.
Preparing to unpack .../jq_1.7.1-3ubuntu0.24.04.2_amd64.deb ...
Unpacking jq (1.7.1-3ubuntu0.24.04.2) ...
Setting up libonig5:amd64 (6.9.9-1build1) ...
Setting up libjq1:amd64 (1.7.1-3ubuntu0.24.04.2) ...
Setting up jq (1.7.1-3ubuntu0.24.04.2) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.9) ...
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q15$ micro api_report.sh
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q15$ chmod +x api_report.sh
./api_report.sh
27.0.0.1 - - [14/Sep/2026 16:32:58] "GET /packages.json HTTP/1.1" 200 -
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q15$ cat summary.md
# Active Packages (downloads ≥ 100)



| Name  | Version | Downloads |
|-------|---------|-----------|
| delta | 0.9.0   | 450       |
| gamma | 1.5.1   | 450       |
| alpha | 1.2.0   | 120       |


```

(a) `python3 -m http.server 8000` 后台启动、`sudo apt install -y jq`

```
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q14$ cd /home/wangbenxi
mkdir q15 && cd q15
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q15$ micro packages.json
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q15$ python3 -m http.server 8000 &
[1] 18896
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q15$ Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
(venv) wangbenxi@WIN-56NAAMF7VQA:~/q15$ sudo apt install -y jq
```

(b) `./api_report.sh` 执行 + `cat summary.md`: 3 个 active 且 downloads ≥ 100 的包，按 downloads 降序、name 升序排列

图 3: q15: curl + jq + Markdown 报告生成

```

1 # src/greetlab/cli.py (临时改坏)
2 def main():
3     p = argparse.ArgumentParser()
4     p.add_argument("--name", required=True)
5     a = p.parse_args()
6     if a.name.isspace():
7         sys.exit(2)
8     print("Hello, name!") # ← 改坏: 字面量而非 f-string

```

./check.sh 因 pytest 失败（或 ruff format check 因引号/换行差异）返回非零。修复后 Makefile:

```

1 .PHONY: check build clean
2
3 check:
4     bash check.sh
5
6 build:
7     python -m build
8
9 clean:
10    rm -rf dist build *.egg-info

```

最终验证与 SHA-256:

```

1 make check          # 通过
2 make build          # 生成 dist/greetlab_25020007117-0.1.0-py3-none-any.whl
3 sha256sum dist/*.whl # c77e45a73e704f0aa98d2c516a334c35474c96fcf3f1b57dfad827cd6a3
4 git add -A && git commit -m "q16 修复并交付"

```

6.3 运行结果与截图

图 4 展示了从临时改坏、check.sh 失败、修复、make check/build 到 SHA-256 计算的完整综合测试。

6.4 结果分析

- **临时改坏与定位:** 把 `print(f"Hello, {a.name}!")` 改成字面量 `print("Hello, name!")` 后, `test_normal_name` 会因输出不匹配失败; 实际流程中先遇到 `ruff format --check` 报 `unformatted: File would be reformatted` (因为引号风格不一致), 用 `ruff format src/greetlab/cli.py` 修复格式问题后, 再通过 `pytest` 定位行为缺陷, 两层防护各司其职;
- **Makefile 简化:** `check` 仅调用 `bash check.sh`, 不重复写三条命令——单一来源原则; `build` 直接跑 `python -m build`; `clean` 清理 `dist`、`build` 和 `*.egg-info`, 比 q14 的 `rm -f` 更彻底 (`-rf` 递归删除目录);

```

=== all checks passed ===
(wenv) wangbenxi@WIN-S6NAHF7VQ4: /q13$ micro Makefile
(wenv) wangbenxi@WIN-S6NAHF7VQ4: /q13$ python3 -m venv venv
source venv/bin/activate
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
pip install build
make check
make build
Writing to /home/wangbenxi/.config/pip/pip.conf
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Collecting build
  Using cached https://pypi.tuna.tsinghua.edu.cn/packages/ad/9b/9fb3585dabdc73a1b2a6267f63f62649347c9e6d872c9fde365b185abb2c/build-1.6.1-py3-none-any.whl (31 kB)
Requirement already satisfied: packaging>=24.0 in /home/wangbenxi/q13/venv/lib/python3.12/site-packages (from build) (26.3)
Collecting pyproject_hooks (from build)
  Using cached https://pypi.tuna.tsinghua.edu.cn/packages/bd/24/12818598c362d7f380f18e74db45963dbc85158324892418c8b49405e42/pyproject_hooks-1.2.0-py3-none-any.whl (18 kB)
Installing collected packages: pyproject_hooks, build
Successfully installed build-1.6.1 pyproject_hooks-1.2.0
./check.sh
=== ruff format check ===
3 files already formatted
=== ruff check ===
All checks passed!
=== pytest ===
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/wangbenxi/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.81s =====
=== all checks passed ===
python -m build
/home/wangbenxi/q16/venv/bin/python: No module named build
make: *** [Makefile:7: build] Error 1
(wenv) wangbenxi@WIN-S6NAHF7VQ4: /q13$ sha256sum dist/*.whl
c77e5a473e704f0aa98d2c516a3344c35474c96fc3f1b573dfad827c8d16a3 dist/greetlab_25020007117-0.1.0-py3-none-any.whl
(wenv) wangbenxi@WIN-S6NAHF7VQ4: /q13$ git add .

```

(a) cp -r q13 q16、临时改坏 cli.py、./check.sh 因 ruff format check 失败、ruff format 修复后通过

```

(wenv) wangbenxi@WIN-S6NAHF7VQ4: /q13$ micro ap1_report.sh
(wenv) wangbenxi@WIN-S6NAHF7VQ4: /q13$ chmod +x ap1_report.sh
./ap1_report.sh
127.0.0.1 - - [14/Sep/2026 16:32:58] "GET /packages.json HTTP/1.1" 200 -
(wenv) wangbenxi@WIN-S6NAHF7VQ4: /q13$ cat summary.md
Active Packages (Downloads > 180)
+-----+-----+
Name | Version | Downloads |
+-----+-----+
delta | 0.9.8 | 458 |
gamma | 1.5.1 | 458 |
alpha | 1.2.0 | 128 |
(wenv) wangbenxi@WIN-S6NAHF7VQ4: /q13$ cd /home/wangbenxi
cp -r q13 q16
cd q16
git init
Initialized existing Git repository in /home/wangbenxi/q16/.git/
(wenv) wangbenxi@WIN-S6NAHF7VQ4: /q13$ micro src/greetlab/cli.py
(wenv) wangbenxi@WIN-S6NAHF7VQ4: /q13$ ./check.sh
=== ruff format check ===
src/greetlab/cli.py:1
1: File would be reformatted
+-----+
def main():
+-----+
1: File would be reformatted, 2 files already formatted
(wenv) wangbenxi@WIN-S6NAHF7VQ4: /q13$ micro src/greetlab/cli.py
(wenv) wangbenxi@WIN-S6NAHF7VQ4: /q13$ ./check.sh
=== ruff format check ===
src/greetlab/cli.py:1
1: File would be reformatted
+-----+
def main():
+-----+
1: File would be reformatted, 2 files already formatted
(wenv) wangbenxi@WIN-S6NAHF7VQ4: /q13$ ruff format src/greetlab/cli.py
1: File reformatted
(wenv) wangbenxi@WIN-S6NAHF7VQ4: /q13$ ./check.sh
=== ruff format check ===
3 files already formatted
=== ruff check ===
All checks passed!
=== pytest ===
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/wangbenxi/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

```

(b) make check 通过、make build 生成 wheel、sha256sum dist/*.whl

图 4: q16: 综合测试——改坏修复、Makefile 与 SHA-256

- **SHA-256**: `sha256sum dist/greetlab_25020007117-0.1.0-py3-none-any.whl` 输出 `c77e45a73e704f0` 用于 wheel 分发后的完整性校验;
- **综合测试反思**: 15分钟时限下, 流程是”初始化 Git → 改坏确认失败 → 修复 → 质量门禁 → 构建 → 计算哈希 → 单一提交”; 实际截图显示 `make build` 首次因 `venv` 未激活 (`python -m build` 找不到模块) 失败, 激活后重试通过, 提示 Makefile 中的 `python` 依赖于当前 shell 的 `PATH`, CI 环境中应显式指定 `venv` 路径。

第 7 章 第 4 周课后练习

课上实验之外, 本周还有 11 道课后练习, 围绕 Make、依赖版本、Git 钩子、CI、正则表达式与静态分析等主题展开。下面用文字加代码的方式逐条记录。

7.1 Makefile 的 clean 目标

`clean` 是最常见的伪目标, 作用就是”撤销构建”, 把生成的产物删掉:

```
.PHONY: clean

clean:
    rm -f paper.pdf
```

这里 `.PHONY` 声明 `clean` 不是一个真正的文件。如果不加这行, 而目录里恰好存在一个叫 `clean` 的文件, `make` 会认为目标已经是最新的, 从而什么都不做。加上之后每次 `make clean` 都会无条件执行删除命令。

7.2 Rust 依赖版本语法的几种场景

`Cargo.toml` 里写依赖版本时有好几种写法, 宽松程度不一样。

尖号 `~1.2.3` 允许更新到 `1.x.x` 里的最新兼容版本, 也就是说主版本号 1 不变, 次版本和补丁版本都可以升。大多数生产项目用这种, 既能自动拿到修复, 又不会被破坏性更新坑到。

波浪号 `~1.2.3` 更保守, 只允许更新到 `1.2.x` 的补丁版本, 次版本号都不动。适合对稳定性要求极高、不想引入任何新功能的场景。

通配符 `1.*` 匹配任意 `1.x.x`, 放得最宽, 适合快速原型开发, 不关心具体小版本。

比较符可以显式指定版本区间, 比如 `>=1.2.3, <2.0.0`, 适合依赖某个 API、同时确定不会升级主版本的情况。还可以同时写多个条件, 比如 `>=1.2, <=1.4`, 满足多个约束, 适合需要兼容特定旧环境、但又不想用太老版本的场景。

7.3 Git pre-commit 钩子: 提交前自动构建

在 `.git/hooks/pre-commit` 里写一个脚本, 让每次提交前先跑一遍构建:

```
#!/bin/sh
make paper.pdf
if [ $? -ne 0 ]; then
```

```
echo "构建失败，提交被拒绝"
exit 1
fi
```

写完记得加可执行权限：

```
chmod +x .git/hooks/pre-commit
```

这样以后每次 `git commit` 之前都会自动构建，构建失败就直接拒绝提交，从源头上避免把编译不过的代码提交上去。

7.4 GitHub Pages、Actions 与自定义 Action

GitHub Pages 最简单：仓库 Settings 里找到 Pages，选择 main 分支的 /root 或 /docs 目录，保存后就会自动发布，访问 <https://用户名.github.io/仓库名> 即可。

然后用 Action 做自动检查。比如对所有 shell 脚本跑 shellcheck，在 `.github/workflows/shellcheck.yml` 里写：

```
name: shellcheck
on: push
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: shellcheck $(find . -name "*.sh")
```

还可以把重复的检查逻辑封装成自定义 Action。比如用 `proselint` 检查 Markdown 文案，在 `.github/actions/prose/action.yml` 里写一个 composite action：

```
name: prose-lint
runs:
  using: composite
  steps:
    - run: pip install proselint && proselint **/*.md
      shell: bash
```

在 workflow 中直接引用本地路径即可调用：

```
- uses: ../github/actions/prose
```

提交一个包含语法问题的 `.md` 文件（比如连续空格、被动语态），Action 就会报错并阻断合并。

7.5 格式化器、linter 与 pre-commit 钩子的配合

以 Python 项目为例，先安装 `black` 和 `ruff`：

```
pip install black ruff
```

然后在 `.git/hooks/pre-commit` 里让提交前先自动格式化、再跑 linter:

```
#!/bin/sh
black .
ruff check . --fix
ruff check .
if [ $? -ne 0 ]; then
    echo "linter 错误未完全修复, 提交被拒绝"
    exit 1
fi
```

`chmod +x .git/hooks/pre-commit` 加上执行权限后钩子就生效了。

让 AI agent 修 linter 错误时, 先让它跑一遍 `ruff check .`, 根据输出逐条修改, 再重新运行, 直到返回码为 0。修完之后人工复查一遍 diff, 确认自动修复只动了风格、没有改变逻辑。

7.6 测试覆盖率与 AI agent

Python 里用 `pytest` 配合 `pytest-cov` 看覆盖率。先写个最简单的例子:

```
1 def add(a, b):
2     return a + b
3
4 def test_add():
5     assert add(1, 2) == 3
```

运行测试并生成 HTML 覆盖率报告:

```
pytest --cov=. --cov-report=html
```

打开 `htmlcov/index.html`, 可以逐行看到哪些行被测试执行过、哪些没覆盖到, 非常直观。

初始覆盖率一般都很低。手动补一批边界测试之后, 可以让 AI agent 接着跑:

```
pytest --cov=. --cov-report=term-missing
```

`term-missing` 会直接在终端列出未覆盖的行号, 让 agent 照着这些行号补测试。AI 生成的测试通常能把普通分支覆盖掉, 但容易忽略异常路径, 有时还会写出没有实际意义的断言, 所以必须人工检查一遍。

7.7 持续集成: GitHub Actions

在 `.github/workflows/ci.yml` 里配一条最基础的流水线, 每次 push 都跑格式检查、linter 和测试:

```
name: ci
on: push
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
```

```

- uses: actions/checkout@v4
- uses: actions/setup-python@v5
  with:
    python-version: "3.11"
- run: pip install black ruff pytest
- run: black --check .
- run: ruff check .
- run: pytest

```

验证流水线是否真的有效，可以故意在代码里写 `import os; os.system("ls")`，或者违反一条 ruff 规则，push 上去之后 CI 变红，就说明门禁确实拦住了问题。

7.8 grep、正则与 semgrep

想找出代码里危险的 `subprocess.Popen` 调用（带 `shell=True` 的），可以直接 grep：

```
grep -Rn 'subprocess\.Popen([^\s]*)shell=True' .
```

但如果正则写得太随意，比如：

```
grep -Rn 'Popen.*shell=True' .
```

这种写法可能漏掉跨行的情况，也处理不了参数之间多个空格的问题。

更可靠的办法是用 semgrep，它基于语法树（AST）而不是文本匹配。写一条规则：

```

rules:
- id: dangerous-popen
  pattern: subprocess.Popen(..., shell=True)
  languages: [python]
  message: "危险的 shell=True"

```

哪怕正则因为格式问题失效，semgrep 依然能准确认出这个危险调用。

7.9 IDE 中的正则查找替换

比如要把 Markdown 里的项目符号从减号统一换成星号。查找框写：

```
^-\s+
```

替换框写：

```
*
```

关键是锚定行首的 `^`，保证只匹配作为项目符号的减号，不会误伤正文中间的连字符或数学里的减号。

7.10 捕获 JSON 中 name 的正则

如果用贪婪匹配：

```
"name": ".*"
```

.* 会一路吃到行尾最后一个引号，把后面的内容也吞进去。

正确的写法是非贪婪地匹配“非引号字符”：

```
"name":\s*"([^"]*)"
```

如果 name 的值里可能出现转义的双引号，还需要支持转义，写成：

```
"name":\s*"((?:\\.|\\\"|\\\\)*)"
```

意思是每一段要么是一个反斜杠加任意字符（转义序列），要么是一个既不是引号也不是反斜杠的普通字符。

7.11 更推荐的做法：直接用 JSON 解析器

再精巧的正则也不如直接用解析器。Python 一行就能取出 name：

```
1 import json, sys
2 print(json.load(sys.stdin)["name"])
```

保存为 extract_name.py，用管道喂一段 JSON 进去：

```
echo '{"name": "Alyssa P. Hacker", "college": "MIT"}' | python extract_name.py
```

输出：

```
Alyssa P. Hacker
```

解析器天然处理转义、空白和嵌套结构，既不用维护复杂正则，也不会出现贪婪匹配的问题。

第 8 章 代码与仓库

项目已托管至 GitHub: <https://github.com/liangbaikai-666/tool-25020007117>

8.1 Git 提交记录

git log --all --graph --decorate --oneline 输出如下：

```
* 5f06c76 更新git log并重新编译report3
* 4d10f5e report3.pdf 编译产物
* dab5537 q09-q12 实验截图
* bd3537f report3.tex 实验报告
* 1d4d1d3 q12 PyTorch线性回归训练
* c804adb q11 协作材料改写
* 4a05b9d q10 清理嵌套目录
* 6a84240 q10 AI驱动修复循环
* d6a240d q09 包结构与源码
* 2b65ccd 第四次作业: q16 重新复制q13并添加Makefile
```

```
* 09f66c7 第四次作业: q13-q16 题目文件预创建
* 296f8e3 第四次作业: q13 check.sh 格式对齐实际输出
* 0a9ea09 第四次作业: q15 api_report.sh 对齐实际输出格式
```

其中 09f66c7 与 296f8e3 为第四次作业的主要提交，覆盖 q13-q16 的源码、Makefile、shell 脚本和测试。

第 9 章 结论

通过本次实验，完成了四道课上实验题，主要收获如下：

- **可执行质量门禁 (q13)**: 建立了 `ruff format --check → ruff check → pytest` 三层串联的 `check.sh`, `set -euo pipefail` 保证任何环节失败即退出; `pyproject.toml` 统一配置 `ruff` 和 `pytest`, `pip install -e .` 解决 `src-layout` 包的 `ModuleNotFoundError`;
- **Make 增量构建 (q14)**: 正确声明 `all → report.txt → stats.txt → data.csv` 依赖链, `.PHONY` 声明 `all` 和 `clean`; 二次 `make` 无改动时 `Nothing to be done`, `touch data.csv` 后联动重建, `make clean` 只删生成物;
- **API + jq + Markdown (q15)**: `python -m http.server 8000` 快速搭建本地 API, `curl -fsS` 适合脚本调用, `jq` 管道用 `-s slurp + select 筛选 + sort_by(-.downloads, .name)` 排序 + `-r raw output` 生成 Markdown 表格;
- **综合测试 (q16)**: 15 分钟内完成”改坏 → 确认失败 → 修复 → 质量门禁 → 构建 → SHA-256 → Git 提交”; Makefile 三目标 (`check / build / clean`), wheel 哈希 `c77e45a7...cd6a3` 用于完整性校验。

总体而言，本实验覆盖了”质量门禁 → 构建系统 → 数据处理 → 综合交付”四个开发工具主题：从让 `check.sh` 成为每次提交前必须跑的门槛，到用 Makefile 精确管理文件依赖避免无效重建，再到用 `curl + jq` 把原始 JSON 转成工程师可读的 Markdown 报告，最后在 15 分钟综合测试中走完”改坏修复 → 构建交付”的完整链路。全部代码、截图与报告已提交至 GitHub (`origin/main`)，便于回溯与复查。